


**VILNIAUS UNIVERSITETAS**

**MATEMATIKOS IR INFORMATIKOS FAKULTETAS**

**INFORMATIKOS STUDIJŲ PROGRAMA**

Laboratorinis darbas

# Realios ir virtualios mašinos projektas paketinei operacinei sistemai

## Real and Virtual Machine Design for a Batch Operating System

Jevgenij Čistiakov

Darbo vadovas : Mantas Grubliauskis

**Vilnius**
**2026**

# Turinys

**Įvadas** . . . . . **3**

**1. Realios mašinos aprašas** . . . . . **4**
1.1. Realios mašinos paskirtis ir bendroji sandara . . . . . 4
1.2. Centrinis procesorius . . . . . 4
1.2.1. Procesoriaus registrai . . . . . 4
1.2.2. Procesoriaus laikas ir taimeris . . . . . 5
1.3. Atmintys . . . . . 6
1.3.1. Naudotojo atmintis . . . . . 6
1.3.2. Supervizorinė atmintis . . . . . 6
1.3.3. Išorinė atmintis . . . . . 6
1.4. Puslapiavimo mechanizmas . . . . . 6
1.5. Pertraukimų mechanizmas . . . . . 7
1.6. Kanalų įrenginys ir duomenų perdavimas . . . . . 8
1.7. Įvedimo ir išvedimo įrenginiai . . . . . 8
1.7.1. Flash atmintinės skaitytuvas . . . . . 8
1.7.2. Spausdintuvas . . . . . 8
1.7.3. Naudotojo sąsaja . . . . . 8
1.8. Realios mašinos bendroji schema . . . . . 9

**2. Virtualios mašinos aprašas** . . . . . **10**
2.1. Virtualios mašinos samprata . . . . . 10
2.2. Virtualios mašinos loginiai komponentai . . . . . 10
2.2.1. VM antraštė . . . . . 10
2.2.2. Virtuali atmintis . . . . . 10
2.2.3. Virtualus procesorius . . . . . 10
2.3. Komandos formatas . . . . . 11
2.4. Virtualios mašinos komandų sistema . . . . . 11
2.4.1. Duomenų perdavimo komandos . . . . . 11
2.4.2. Aritmetinės komandos . . . . . 12
2.4.3. Loginės ir palyginimo komandos . . . . . 12
2.4.4. Valdymo perdavimo komandos . . . . . 12
2.4.5. Įvedimo/išvedimo ir sisteminės komandos . . . . . 13
2.5. Įvedimo/išvedimo mechanizmas . . . . . 13
2.6. Programos ir užduoties tekstinis formatas . . . . . 13
2.7. Virtualios ir realios mašinos komponentų sąryšis . . . . . 15

**3. Virtuali mašina operacinės sistemos kontekste** . . . . . **16**

# Įvadas

Šio laboratorinio darbo tikslas – suprojektuoti realios ir virtualios mašinos modelį paketinei operacinei sistemai. Darbe aprašoma realios mašinos techninė įranga, virtualios mašinos loginiai komponentai, virtualios mašinos komandų sistema, atminties puslapiavimas, pertraukimai ir įvedimo/išvedimo mechanizmas. Taip pat pateikiamas tekstinis užduoties formatas bei trumpai paaiškinama virtualios mašinos vieta visos operacinės sistemos kontekste.

Modelis sąmoningai laikomas paprastas. Mokomosios operacinės sistemos tikslas yra parodyti OS darbo principus, todėl nėra siekiama atkartoti konkretaus šiuolaikinio procesoriaus ar jo instrukcijų rinkinio. Toks požiūris atitinka G. Šiaulio mokomosios operacinės sistemos modelį, kuriame reali ir virtuali mašinos apibrėžiamos tiek detaliai, kiek reikia OS mechanizmams paaiškinti [Šia03].

Pagal laboratorinio darbo variantą projektuojama paketinė OS. Virtuali mašina turi 16 blokų po 16 žodžių, vieno žodžio ilgis šiame projekte yra 4 baitai. Virtualus procesorius projektuojamas registrinės architektūros: pagrindiniai duomenų registrai yra `AX` ir `BX`. Įvedimo įrenginys – realios mašinos flash atmintinės skaitytuvas, išvedimo įrenginys – spausdintuvas, o išorinė atmintis – kietasis diskas.

# 1. Realios mašinos aprašas

## 1.1. Realios mašinos paskirtis ir bendroji sandara

Reali mašina (RM) – tai projektuojamos paketinės operacinės sistemos aparatinės įrangos modelis. Ji turi vieną centrinį procesorių, naudotojo atmintį, supervizorinę atmintį, puslapiavimo mechanizmą, kanalų įrenginį, įvedimo ir išvedimo įrenginius bei išorinę atmintį. Tokia sandara artima mokomosios OS modeliui, pateiktam G. Šiaulio darbe: OS slepia realios mašinos detales ir naudotojo užduočiai pateikia paprastesnę virtualią mašiną [Šia03].

RM žodis yra 4 baitų (32 bitų) ilgio. Viename žodyje gali būti laikomas sveikasis skaičius su ženklu, keturi simboliai, adresavimo informacija arba viena virtualios mašinos komanda. Naudotojo atmintis skirstoma į blokus po 16 žodžių, nes vienas toks blokas atitinka vieną virtualios mašinos puslapį.

## 1.2. Centrinis procesorius

Procesoriaus paskirtis – vykdyti virtualios mašinos komandas ir operacinės sistemos kodą. Procesorius dirba dviem režimais:

- **USER** – vykdomos naudotojo programos VM komandos;
- **SUPERVISOR** – vykdomos operacinės sistemos operacijos, pertraukimų apdorojimas, puslapiavimo ir įvedimo/išvedimo valdymas.

Naudotojo režime programa negali tiesiogiai keisti `PTR`, `MODE`, `PI`, `SI` ir `TI` registrų ar tiesiogiai valdyti realių įrenginių. Privilegijuotas veiksmas atliekamas per supervizorinį pertraukimą. Toks dviejų režimų modelis leidžia atskirti naudotojo programą nuo OS valdymo informacijos.

### 1.2.1. Procesoriaus registrai

| Registras | Dydis | Paskirtis |
|---|---|---|
| `AX` | 32 bitų | Pirmasis pagrindinis duomenų registras. Jame laikomas pirmasis aritmetinių ir loginių operacijų operandas bei rezultatas. |
| `BX` | 32 bitų | Antrasis pagrindinis duomenų registras. Jame laikomas antrasis operandas. |
| `PC` | 16 bitų | Programos skaitiklis. Naudojama virtualaus adreso dalis 0–255; registras nurodo kitą vykdomą VM komandą. |
| `FLAGS` | loginis | Požymių registras. Naudojami `ZF`, `SF` ir `OF` požymiai palyginimui ir sąlyginiams šuoliams. |

| Registras | Dydis | Paskirtis |
|---|---|---|
| PTR | 32 bitų | Puslapių lentelės registras. Nurodo proceso puslapių lentelę ir saugo informaciją apie naudojamų bei maksimalių puslapių skaičių. |
| MODE | 1 bitas | Procesoriaus darbo režimas: USER arba SUPERVISOR. |
| PI | loginis kodas | Programinio pertraukimo registras. Nurodo klaidą, aptiktą vykdant VM komandą. |
| SI | loginis kodas | Supervizorinio pertraukimo registras. Naudojamas VM prašymams, kuriuos gali atlikti tik OS. |
| TI | sveikasis | Taimerio registras. Saugo einamajai VM likusį procesoriaus taktų skaičių. |

***1 lentelė.*** *Realios mašinos procesoriaus registrai.*

**Požymių registras.** Šiame modelyje naudojamas tik toks požymių rinkinys, kurio reikia pasirinktam komandų rinkiniui:

- ZF (Zero Flag) – 1, jei rezultatas lygus nuliui;
- SF (Sign Flag) – rodo rezultato ženklą;
- OF (Overflow Flag) – 1, jei atliekant operaciją su 32 bitų sveikaisiais skaičiais su ženklu įvyksta perpildymas.

Požymių bitų fizinis išdėstymas šiame projektavimo etape nefiksuojamas, nes jis nėra svarbus VM logikai.

### 1.2.2. Procesoriaus laikas ir taimeris

Taimerio mechanizmas naudojamas tam, kad viena užduotis negalėtų neribotai laikyti procesoriaus. Kai VM gauna procesorių, registrui TI priskiriama pasirinkta laiko kvanto reikšmė $N$. Vykdant komandą TI mažinamas jos trukme:

- įprasta atminties, aritmetinė, loginė ar valdymo perdavimo komanda – 1 taktas;
- įvedimo arba išvedimo komanda – 3 taktai.

Kai TI=0, fiksuojamas taimerio pertraukimas ir valdymas grąžinamas operacinei sistemai. Toks principas tiesiogiai atitinka mokomosios OS taimerio mechanizmą [Šia03]. Konkreti $N$ reikšmė yra OS planavimo parametro dalis ir šiame projektavimo etape nefiksuojama.

## 1.3. Atmintys

### 1.3.1. Naudotojo atmintis

Naudotojo atmintis skirta virtualių mašinų puslapiams ir jų puslapių lentelėms laikyti. Ji organizuojama blokais po 16 žodžių. Kiekvienas VM virtualios atminties puslapis atvaizduojamas į vieną laisvą naudotojo atminties bloką. Vienos VM puslapiai neprivalo būti gretimi realioje atmintyje.

Bendras fizinių naudotojo atminties blokų skaičius šiame projektavimo etape nėra fiksuojamas: tai realizacijos parametras. Svarbu, kad OS galėtų skirti virtualiai mašinai reikalingus puslapius ir vieną bloką jos puslapių lentelei.

### 1.3.2. Supervizorinė atmintis

Supervizorinė atmintis skirta OS informacijai, kuri negali būti tiesiogiai prieinama naudotojo programai. Joje logiškai laikomi:

- operacinės sistemos kodas ir sisteminiai kintamieji;
- pertraukimų apdorojimo informacija;
- procesų ir resursų valdymo duomenys;
- įvedimo/išvedimo buferiai ir kanalų valdymo informacija.

Kaip ir G. Šiaulio mokomajame modelyje, supervizorinė atmintis gali būti realizuojama aukšto lygio kalbos duomenų struktūromis, todėl jos konkretus fizinis dydis šiame projekto etape nėra esminis [Šia03].

### 1.3.3. Išorinė atmintis

Išorinė atmintis – kietasis diskas. Jis naudojamas paketinių užduočių saugojimui. Naudotojas pateikia užduočių paketą flash atmintinėje, OS patikrina kiekvienos užduoties sintaksę ir teisingas užduotis perkelia į diską. Kai atsiranda reikalingų resursų, užduotis iš disko įkeliama į naudotojo atmintį ir jai sukuriama VM.

Diskas modeliuojamas kaip blokinė atmintis. Vieno loginio bloko dydis parenkamas 16 žodžių, kad duomenys būtų patogiai perduodami tais pačiais blokais kaip ir VM puslapiai. Bendras disko blokų skaičius yra realizacijos parametras.

## 1.4. Puslapiavimo mechanizmas

Virtuali mašina turi 16 puslapių po 16 žodžių, todėl jos virtualus adresas sudarytas iš dviejų dalių: puslapio numerio $p$ ir žodžio poslinkio $d$. Abu dydžiai yra intervale 0–15. Virtualus adresas apskaičiuojamas:

$$A_v = 16p + d.$$

Kiekviena VM turi puslapių lentelę iš 16 įrašų. Įraše $PT[p]$ saugomas realaus naudotojo atminties bloko numeris, kuriame yra virtualus puslapis $p$. Tada fizinis žodžio adresas yra:

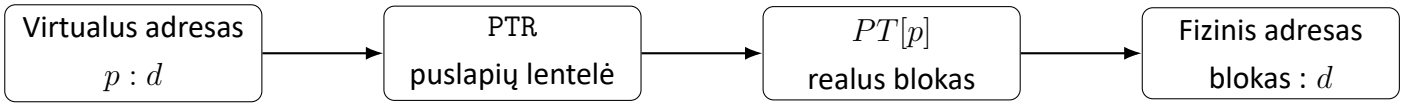
$$A_f = 16 \cdot PT[p] + d.$$

Puslapių lentelė užima vieną 16 žodžių naudotojo atminties bloką ir nėra įtraukiama į VM virtualių adresų erdvę, todėl naudotojo programa jos tiesiogiai pasiekti negali. Registras `PTR` nurodo šios lentelės vietą. Toks virtualių ir realių adresų susiejimas atitinka G. Šiaulio aprašytą puslapiavimo principą [Šia03].

**PTR formatas.** `PTR` yra 4 baitų registras, simboliškai žymimas $a_0a_1a_2a_3$:

- $a_0$ – šiuo metu programai skirtų virtualių puslapių skaičius;
- $a_1$ – maksimalus VM puslapių skaičius, šiame variante $a_1 = 16$;
- $a_2a_3$ – puslapių lentelės bazinio realaus adreso dalis.

Taip įgyvendinamas papildomas reikalavimas puslapių registre saugoti programos dydį ir maksimalų puslapių skaičių.




***1 pav.*** *Virtualaus adreso vertimo principas.*

## 1.5. Pertraukimų mechanizmas

Po kiekvienos VM komandos procesorius patikrina `PI`, `SI` ir `TI`. Jei bent viena pertraukimo priežastis aktyvi, VM vykdymas sustabdomas, procesorius pereina į SUPERVISOR režimą ir OS apdoroja priežastį. Mokomajame modelyje pertraukimas suprantamas kaip procesoriaus būsenos pasikeitimas, kurį OS turi aptikti ir apdoroti [Šia03].

| Registras | Kodas | Reikšmė |
|---|---|---|
| PI | 1 | neteisingas virtualus adresas |
| PI | 2 | neteisingas operacijos kodas |
| PI | 3 | neleistinas priskyrimas arba apsaugos pažeidimas |
| PI | 4 | aritmetinis perpildymas |
| PI | 5 | dalyba iš nulio |
| SI | 1 | VM prašo įvedimo iš flash įrenginio |
| SI | 2 | VM prašo išvedimo į spausdintuvą |
| SI | 3 | VM prašo sukurti proceso kopiją (`FORK`) |
| SI | 4 | VM programa baigė darbą (`HALT`) |

***2 lentelė.*** *Naudojamos programinių ir supervizorinių pertraukimų priežastys.*

Taimerio pertraukimas fiksuojamas atskirai, kai TI=0.

## 1.6. Kanalų įrenginys ir duomenų perdavimas

Kanalų įrenginys naudojamas blokiniam duomenų perdavimui tarp atminčių ir I/O įrenginių. Jį valdyti galima tik SUPERVISOR režime. Sąsaja sąmoningai paliekama paprasta ir paremta G. Šiaulio modeliu [Šia03]:

- ST – šaltinio tipas: naudotojo atmintis, supervizorinė atmintis, kietasis diskas arba flash įvesties srautas;
- DT – paskirties tipas: naudotojo atmintis, supervizorinė atmintis, kietasis diskas arba spausdintuvas;
- SB – šaltinio bloko numeris;
- DB – paskirties bloko numeris.

Nustačiusi šiuos registrus OS inicijuoja komandą XCHG, kuri perkelia vieną 16 žodžių bloką. Kanalų įrenginys šiame modelyje neturi atskiro procesoriaus – perdavimą valdo centrinis procesorius. Toks aprašas nusako sąsają, bet neįpareigoja konkrečiai programinei realizacijai.

## 1.7. Įvedimo ir išvedimo įrenginiai

### 1.7.1. Flash atmintinės skaitytuvas

Flash atmintinės skaitytuvas yra **realios mašinos įvedimo įrenginys**. Naudotojas per jį pateikia paketą, kuriame yra viena ar kelios užduotys. OS nuskaito paketą, tikrina užduočių sintaksę ir teisingas užduotis perkelia į kietąjį diską. Flash įrenginys nėra atskiras virtualios mašinos aparatinis komponentas.

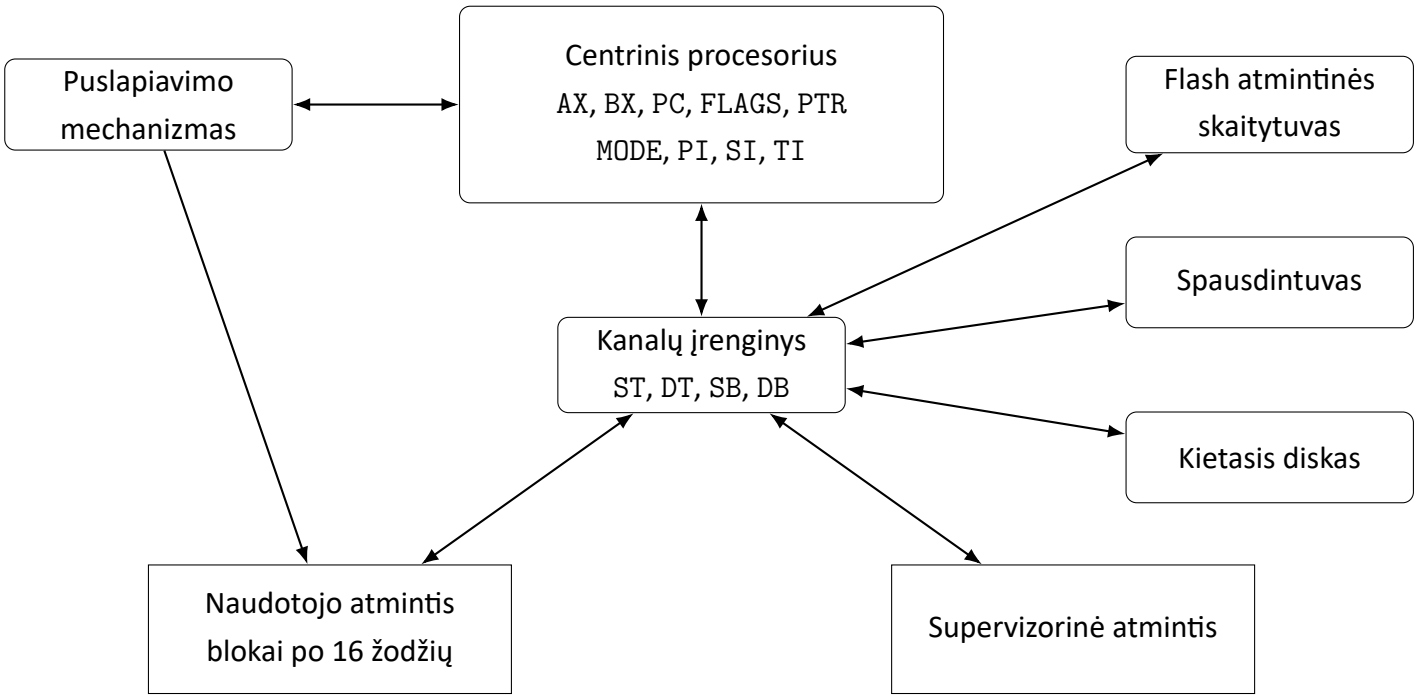
### 1.7.2. Spausdintuvas

Spausdintuvas yra realios mašinos išvedimo įrenginys. VM gali tik pateikti išvedimo prašymą. Fizinį perdavimą į spausdintuvą atlieka OS SUPERVISOR režime per kanalų įrenginį.

### 1.7.3. Naudotojo sąsaja

Kadangi projektuojama paketinė OS, naudotojo sąsaja nėra interaktyvus VM langas. Naudotojas pateikia užduočių paketą flash atmintinėje, o rezultatus gauna per spausdintuvą. Taigi įvedimo srautas yra užduočių paketas, o išvedimo srautas – OS ir VM programų suformuoti spausdinimo duomenys. Konkretus grafinis ar konsolinis UI šiame projektavimo etape nėra būtinas.

## 1.8. Realios mašinos bendroji schema




***2 pav.*** *Projektuojamos realios mašinos komponentai ir jų sąveika.*

Schema parodo pagrindinį principą: USER režime vykdoma VM programa dirba tik su jai pateikta virtualia atmintimi ir procesoriaus registrais, o fizinius adresus bei I/O įrenginius valdo OS SUPERVISOR režime.

# 2. Virtualios mašinos aprašas

## 2.1. Virtualios mašinos samprata

Virtuali mašina (VM) – tai naudotojo užduočiai pateikiamas supaprastintas realios mašinos vaizdas. Kiekviena vykdoma užduotis turi savo VM ir gali elgtis taip, tarsi turėtų atskirą procesorių bei atmintį. Realius procesoriaus, atminties ir I/O resursus paskirsto operacinė sistema. Toks principas leidžia atskirti naudotojo programą nuo realios mašinos techninių detalių [Šia03].

Šiame projekte VM yra registrinės architektūros. Pagrindiniai duomenų registrai yra `AX` ir `BX`; aritmetinės ir loginės komandos operuoja šiais registrais.

## 2.2. Virtualios mašinos loginiai komponentai

### 2.2.1. VM antraštė

Kuriant VM jai priskiriama antraštė (*header*). Minimalus antraštės laukas yra **ID** – unikalus virtualios mašinos identifikatorius. Pradinės užduoties ID gaunamas iš užduoties failo antraštės. Jei vykdant `FORK` sukuriama nauja VM kopija, OS vaikinės VM antraštei priskiria naują ID.

VM antraštė yra OS valdymo informacija ir nėra naudotojo programos virtualios atminties dalis.

### 2.2.2. Virtuali atmintis

VM atmintį sudaro 16 blokų po 16 žodžių. Iš viso VM turi:

$$16 \cdot 16 = 256 \text{ žodžių.}$$

Vienas žodis yra 4 baitų, todėl maksimali virtuali adresų erdvė lygi 1024 baitams. Virtualūs adresai yra `0x00`–`0xFF`. Pirmasis šešioliktainis skaitmuo nurodo puslapį, antrasis – žodį puslapyje. Pavyzdžiui, adresas `0x3A` reiškia 3 puslapio 10 žodį.

Toje pačioje virtualioje adresų erdvėje laikomi programos duomenys ir kodas. Kur šios sritys prasideda, nurodo užduoties failo direktyvos `DATASEG` ir `CODESEG`.

### 2.2.3. Virtualus procesorius

Virtualus procesorius turi tik tuos registrus, kuriuos tiesiogiai mato naudotojo programa:

| Registras | Dydis | Paskirtis |
|---|---|---|
| AX | 32 bitų | Pirmasis pagrindinis duomenų registras ir aritmetinių/loginių operacijų rezultato registras. |
| BX | 32 bitų | Antrasis pagrindinis duomenų registras. |
| PC | 16 bitų | Programos skaitiklis; naudojamos reikšmės 0–255. |
| FLAGS | loginis | Požymiai ZF, SF, OF, kuriuos naudoja palyginimo ir sąlyginio valdymo perdavimo komandos. |

***3 lentelė.*** *Virtualaus procesoriaus registrai.*

Virtualus procesorius neturi tiesioginės prieigos prie PTR, MODE, PI, SI ir TI: tai realios mašinos ir OS valdymo registrai.

## 2.3. Komandos formatas

Vienai VM komandai skiriamas vienas 4 baitų atminties žodis. Kai komandai reikia adreso, naudojamas dviejų simbolių operacijos kodas ir du šešioliktainiai adreso simboliai. Pavyzdžiui:

LA20, SB3A, JG50.

Komandoms be adreso visi keturi baitai neprivalo būti užimti. Tekstiniame faile jos rašomos įprastu mnemonic'u, pavyzdžiui ADD, DIV, FORK, HALT. Konkretus vidinis bitų išdėstymas nėra šio laboratorinio darbo tikslas – svarbi yra komandos semantika.

## 2.4. Virtualios mašinos komandų sistema

Komandos suskirstytos pagal paskirtį: duomenų perdavimo, aritmetinės, loginės ir palyginimo, valdymo perdavimo, įvedimo/išvedimo bei sisteminės.

### 2.4.1. Duomenų perdavimo komandos

| Komanda | Paskirtis |
|---|---|
| LAxy | Įkelia virtualios atminties žodį adresu xy į AX: $AX := [xy]$. |
| LBxy | Įkelia virtualios atminties žodį adresu xy į BX: $BX := [xy]$. |
| SAxy | Įrašo AX reikšmę į virtualios atminties žodį xy: $[xy] := AX$. |
| SBxy | Įrašo BX reikšmę į virtualios atminties žodį xy: $[xy] := BX$. |

***4 lentelė.*** *Duomenų perdavimo komandos.*

Adresas xy interpretuojamas kaip $16x + y$, kur $x$ yra puslapio numeris, o $y$ – žodžio poslinkis puslapyje.

### 2.4.2. Aritmetinės komandos

| Komanda | Paskirtis |
|---|---|
| `ADD` | $AX := AX + BX$. Rezultatas paliekamas `AX`. |
| `SUB` | $AX := AX - BX$. Rezultatas paliekamas `AX`. |
| `MUL` | $AX := AX \cdot BX$. Rezultatas paliekamas `AX`. |
| `DIV` | $AX := AX/BX$, naudojama sveikoji dalis. Jei $BX = 0$, nustatomas programinis pertraukimas. |

***5 lentelė.*** *Aritmetinės komandos.*

Po operacijų atnaujinami reikalingi `ZF`, `SF` ir `OF` požymiai. Jei rezultatas netelpa į 32 bitų skaičių su ženklu diapazoną, nustatomas `OF` ir gali būti fiksuojamas `PI=4`.

### 2.4.3. Loginės ir palyginimo komandos

Loginės operacijos įtrauktos kaip papildomas komandų grupės išplėtimas, tačiau jų skaičius sąmoningai ribotas.

| Komanda | Paskirtis |
|---|---|
| `AND` | $AX := AX$ & $BX$. |
| `OR` | $AX := AX \mid BX$. |
| `XOR` | $AX := AX \oplus BX$. |
| `CMP` | Apskaičiuoja požymius taip, lyg būtų atlikta $AX - BX$, tačiau registrų reikšmių nekeičia. |

***6 lentelė.*** *Loginės ir palyginimo komandos.*

### 2.4.4. Valdymo perdavimo komandos

Valdymo perdavimo komandų rinkinys paliekamas minimalus: vienas besąlyginis ir trys sąlyginiai šuoliai.

| Komanda | Sąlyga | Veiksmas |
|---|---|---|
| `JMxy` | visada | $PC := xy$. |
| `JExy` | $ZF = 1$ | Šuolis, kai po `CMP` $AX = BX$. |
| `JGxy` | $ZF = 0$ ir $SF = OF$ | Šuolis, kai $AX > BX$ lyginant skaičius su ženklu. |
| `JLxy` | $SF \neq OF$ | Šuolis, kai $AX < BX$ lyginant skaičius su ženklu. |

***7 lentelė.*** *Valdymo perdavimo komandos.*

### 2.4.5. Įvedimo/išvedimo ir sisteminės komandos

| Komanda | Paskirtis |
|---|---|
| GDx0 | Paprašo OS nuskaityti vieną 16 žodžių bloką iš realios mašinos flash įvedimo srauto į VM puslapį x. Nustatomas SI=1. |
| PDx0 | Paprašo OS išvesti VM puslapį x į realios mašinos spausdintuvą. Nustatomas SI=2. |
| FORK | Paprašo OS sukurti einamosios VM/proceso kopiją. Tėviniame procese AX gauna naujos vaikinės VM ID, vaikiniame procese AX=0. Nustatomas SI=3. |
| HALT | Užbaigia naudotojo programos vykdymą. Nustatomas SI=4. |

***8 lentelė.*** *Įvedimo/išvedimo ir sisteminės komandos.*

## 2.5. Įvedimo/išvedimo mechanizmas

VM neturi savo fizinio flash skaitytuvo ar spausdintuvo. Ji turi tik I/O komandas, kurios perduoda prašymą operacinei sistemai. Pavyzdžiui, vykdant GD30:

1. VM komandos interpretatorius nustato SI=1 ir nurodo virtualų puslapį 3;
2. procesorius pereina į SUPERVISOR režimą;
3. OS pagal PTR ir puslapių lentelę nustato realų bloką, atitinkantį VM puslapį 3;
4. OS sukonfigūruoja kanalų įrenginį: šaltinis – realios mašinos flash įvesties srautas, paskirtis – atitinkamas naudotojo atminties blokas;
5. atliekama XCHG operacija ir valdymas grąžinamas VM.

PDx0 vykdoma analogiškai, tik duomenys perduodami iš VM puslapio į realios mašinos spausdintuvą. Tokiu būdu išlaikoma aiški riba: flash ir spausdintuvas priklauso realiai mašinai, o VM juos pasiekia tik per OS.

## 2.6. Programos ir užduoties tekstinis formatas

Kadangi projektuojama paketinė OS, flash atmintinėje gali būti pateikiamos kelios užduotys. Kiekviena užduotis turi antraštę, duomenų segmentą, kodo segmentą ir pabaigos žymę. Vienas galimas tekstinis formatas:

```
$JOB
ID 0001
PAGES 02
DATASEG 00
DW 20
```

```
DW 30
DW 60
DW -1000
DW 0
DW 0
CODESEG 10
LA00
LB01
MUL
SA05
LA02
LB00
DIV
LB05
ADD
LB03
SUB
SA04
PD00
HALT
$END
```

Ši programa apskaičiuoja išraišką

$$(20 \cdot 30 + 60/20) - (-1000)$$

ir rezultatą įrašo į duomenų žodį adresu `0x04`.

Direktyvų reikšmės:

**`$JOB.`** pradeda užduoties antraštę;

**`ID nnnn.`** nurodo užduoties/VM identifikatorių;

**`PAGES n.`** nurodo pradžioje reikalingų virtualių puslapių skaičių; ši reikšmė gali būti įrašoma į `PTR` lauką $a_0$, o $a_1 = 16$;

**`DATASEG aa.`** nurodo duomenų segmento pradžios virtualų adresą;

**`DW x.`** įrašo vieną 32 bitų sveikąjį skaičių su ženklu;

**`DB abcd.`** į vieną žodį įrašo iki keturių simbolių;

**`CODESEG aa.`** nurodo kodo segmento pradžią; šia reikšme inicializuojamas `PC`;

**`$END.`** žymi užduoties pabaigą.

Prieš perkeldama užduotį į kietąjį diską OS patikrina bent šias sąlygas:

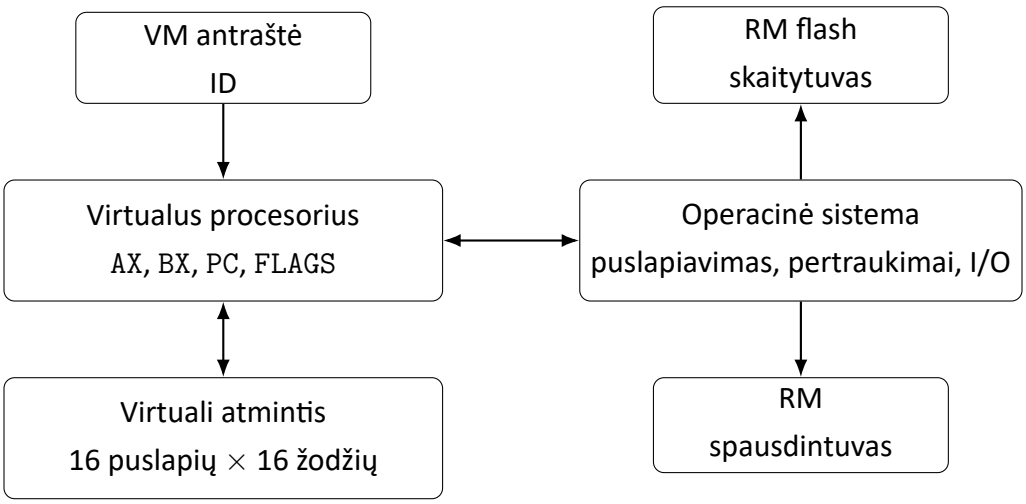
- yra antraštė, nurodytas ID ir jis nesikartoja tame pačiame pakete;
- yra `DATASEG`, `CODESEG` ir `$END`;
- komandų mnemonicai priklauso apibrėžtai VM komandų sistemai;
- naudojami adresai patenka į `0x00`–`0xFF`;
- deklaruotas puslapių skaičius yra 1–16;
- duomenų ir kodo sritys nesikerta;
- programoje yra `HALT` komanda.

Šiame projekte nereikalaujama atskiro kompiliatoriaus. Įkėlimo modulis gali tiesiog interpretuoti tekstinį formatą ir sudėti 4 baitų VM komandas bei duomenis į nurodytas virtualios atminties vietas.

## 2.7. Virtualios ir realios mašinos komponentų sąryšis

| VM komponentas | Sąryšis su realia mašina |
|---|---|
| VM antraštė ir ID | Laikomi OS valdymo informacijoje; pagal ID identifikuojama konkreti VM. |
| Virtualus procesorius | `AX`, `BX`, `PC` ir `FLAGS` reikšmės vykdant VM naudojamos realiame procesoriuje, o keičiant vykdomą VM išsaugomos OS. |
| Virtuali atmintis | VM puslapiai laikomi realios mašinos naudotojo atminties blokuose. Virtualų adresą į realų verčia puslapiavimo mechanizmas pagal `PTR`. |
| Duomenų ir aritmetinės komandos | Interpretuojamos realaus procesoriaus USER režime. |
| I/O komandos | Sukelia `SI`; tikrą flash skaitytuvą arba spausdintuvą valdo OS SUPERVISOR režime per kanalų įrenginį. |
| `FORK` ir `HALT` | Sukelia supervizorinį pertraukimą; veiksmą atlieka OS. |
| VM klaidos | Neteisinga komanda, adresas ar aritmetinė klaida nustato `PI`; klaidą apdoroja OS. |

***9 lentelė.*** *VM loginių komponentų sąryšis su realios mašinos komponentais.*




***3 pav.*** *Virtualios mašinos modelis ir I/O ryšys per operacinę sistemą.*

# 3. Virtuali mašina operacinės sistemos kontekste

Projektuojama OS yra paketinė. Naudotojas nepaleidžia kiekvienos programos interaktyviai: jis pateikia flash atmintinėje paruoštą vienos ar kelių užduočių paketą. Operacinė sistema valdo visą užduoties kelią nuo įvedimo iki rezultato išvedimo.

Bendras užduoties apdorojimas yra toks:

1. naudotojas pateikia užduočių paketą realios mašinos flash atmintinės skaitytuvui;
2. OS nuskaito užduoties antraštę, ID, duomenų ir kodo dalis bei patikrina sintaksę;
3. teisinga užduotis įrašoma į kietąjį diską ir laukia resursų;
4. atsiradus laisvos naudotojo atminties, OS paskiria reikalingus puslapius, sudaro puslapių lentelę, nustato `PTR`, sukuria VM antraštę su ID ir inicializuoja `AX`, `BX`, `PC` bei `FLAGS`;
5. VM vykdoma USER režimu; I/O, `FORK`, `HALT`, klaidos ir taimerio pabaiga perduoda valdymą OS;
6. programai pasibaigus OS atlaisvina jos resursus ir gali pradėti kitą diske laukiančią užduotį.

Komanda `FORK` yra VM prašymas operacinei sistemai sukurti einamosios VM kopiją. Nauja VM gauna naują ID ir atskirą procesoriaus būseną bei virtualios atminties atvaizdavimą. Tėviniame procese į `AX` grąžinamas vaikinės VM ID, o vaikiniame procese `AX`=0, todėl ta pati programa gali atskirti abi vykdymo šakas.

Taigi virtuali mašina šiame projekte yra naudotojo programos vykdymo aplinka, o OS yra tarpininkas tarp VM ir realios mašinos. VM mato paprastą procesorių su `AX`, `BX`, `PC` ir `FLAGS`, 256 žodžių virtualią atmintį bei I/O komandas. Operacinė sistema valdo realią atmintį, puslapiavimą, taimerį, flash skaitytuvą, spausdintuvą, kietąjį diską ir užduočių vykdymo tvarką.

# Literatūra

[Šia03] G. Šiaulys. „Mokomoji operacinė sistema“. Magistro darbas. Vilniaus universitetas, Matematikos ir informatikos fakultetas, 2003.